

Python 单选题库

一、python 语法基础

1、 Python 3.x 版本的保留字总数是

- A.27 B.29 C.33 D.16

2. 以下选项中，不是 Python 语言保留字的是

- A while B pass C do D except

3. 关于 Python 程序格式框架，以下选项中描述错误的是

A Python 语言不采用严格的“缩进”来表明程序的格式框架

B Python 单层缩进代码属于之前最邻近的一行非缩进代码,多层缩进代码根据缩进关系决定所属范围

C Python 语言的缩进可以采用 Tab 键实现

D 判断、循环、函数等语法形式能够通过缩进包含一批 Python 代码，进而表达对应的语义

4. 下列选项中不符合 Python 语言变量命名规则的是

- A TempStr B I C 3_1 D _AI

5. 以下选项中，关于 Python 字符串的描述错误的是

A Python 语言中，字符串是用一对双引号”” 或者一对单引号 ’ ’ 括起来的零个或者多个字符

B 字符串包括两种序号体系：正向递增和反向递减

C 字符串是字符的序列，可以按照单个字符或者字符片段进行索引

D Python 字符串提供区间访问方式，采用 [N:M] 格式，表示字符串中从 N 到 M 的索引子字符串（包含 N 和 M）

6. 给出如下代码

```
TempStr="Hello World"
```

可以输出 “World” 子串的是

- A 。 print (TempStr[-5:0]) B print(TempStr [-5:])
- C print (TempStr [-5: -1]) D print(TempStr [-4: -1])

7. 关于赋值语句,以下选项中描述错误的是

A a,b = b,a 可以实现 a 和 b 值的互换

B a,b, c = b,c,a 是不合法的

C 在 Python 语言中，“=” 表示赋值，即将 “=” 右侧的计算结果赋值给左侧变量，包含 “=” 的语句称为赋值语句

D 赋值与二元操作符可以组合，例如 &=

8. 关于 eval 函数,以下选项中描述错误的是

A eval 函数的定义为: eval (source, globals=None, locals=None, /)

B 执行 “>>> eval("Hello") ” 和执行 “>>> eval (”” Hello'”) ” 得到相同的结果

C eval 函数的作用是将输入的字符串转为 Python 语句，并执行该语句

D 如果用户希望输入一个数字，并用程序对这个数字进行计算，可以采用 eval (input (<输入提示字符串>))组合

9. 关于 Python 语言的注释,以下选项中描述错误的是

A Python 语言有两种注释方式：单行注释和多行注释

B Python 语言的单行注释以 # 开头

C Python 语言的多行注释以 ’ ’ ’ （三个单引号）开头和结尾

D Python 语言的单行注释以单引号 ’ 开头

10. 关于 Python 语言的特点,以下选项中描述错误的是

- A Python 语言是脚本语言 B Python 语言是非开源语言
C Python 语言是跨平台语言 D Python 语言是多模型语言

11. 关于 import 引用, 以下选项中描述错误的是

- A 可以使用 from turtle import setup 引入 turtle 库
B 使用 import turtle as t 引入 turtle 库, 取别名为 t
C 使用 import turtle 引入 turtle 库
D import 保留字用于导入模块或者模块中的对象

12. 下面代码的输出结果是

```
print(0.1+0.2==0.3)
```

- A false B True C False D true

13. 下面代码的输出结果是

```
print(round(0.1+0.2,1) == 0.3)
```

- A 0 B 1 C False D True

14. 在一行上写多条 Python 语句使用的符号是

- A 点号 B 冒号 C 分号 D 逗号

15. 给出如下代码

```
s = 'Python is beautiful!'
```

可以输出 “python” 的是

- A print (s [0:6].lower ()) B print(s[: -14])
C print (s [0:6]) D print (s[-21: -14].lower)

16. 给出如下代码

```
s = 'Python is Open Source!'
```

```
print (s [0: ] .upper() )
```

上述代码的输出结果是

- A PYTHON B PYTHON IS OPEN SOURCE
C Python is Open Source! D PYTHON IS OPEN SOURCE !

17. 以下选项中, 符合 Python 语言变量命名规则的是

- A Templist B ! 1 C (VR) D 5_1

18. 下列选项中可以准确查看 Python 代码的语言版本 (3.5.3) 的是

- A >> > import sys B >> > import sys
 >> > sys。version >> > sys.exc_info ()
C >>> import sys D >>> import sys
 >> > sys.version—info >>> sys。path

19. 下列选项中可以获取 Python 整数类型帮助的是

- A >>> help(float) B > > dir (str) C >>> help (int) D >>> dir (int)

20. 给出如下代码:

```
>> > x = 3.14
```

```
>> > eval('x + 10')
```

上述代码的输出结果是

- A TypeError: must be str, not int B 系统 错
C 13.14 D 3.1410

21. Python 语言的主网站网址是

- A https: //www.python123.org/ B https: //www。python.org/

- C <https://www.python123.io/> D <https://pypi.python.org/pypi>
22. 下列 Python 保留字中, 用于异常处理结构中用来捕获特定类型异常的是
A def B except C while D pass
23. 关于 Python 注释, 以下选项中描述错误的是
A Python 注释语句不被解释器过滤掉, 也不被执行
B 注释可用于标明作者和版权信息
C 注释可以辅助程序调试
D 注释用于解释代码原理或者用途
24. 以下选项中, 不是 Python 数据类型的是
A 实数 B 列表 C 整数 D 字符串
25. 下列 Python 保留字中, 不用于表示分支结构的是
A elif B in C if D else
26. 以下选项中, 不属于 Python 保留字的是
A def B import C type D elif
27. 以下选项中, 对程序的描述错误的是
A 程序是由一系列函数组成的 B 通过封装可以实现代码复用
C 可以利用函数对程序进行模块化设计 D 程序是由一系列代码组成的
28. 利用 print() 格式化输出, 能够控制浮点数的小数点后两位输出的是
A {:.2} B {:.2f} C {:.2} D {:.2f}
29. 以下选项中可用作 Python 标识符的是
A 3B9909 B class C ____ D it's
30. 关于 Python 赋值语句, 以下选项中不合法的是
A x=(y=1) B x,y=y, x C x=y=1 D x=1;y=1
31. 以下选项中, 不是 Python 语言保留字的是
A int B del C try D None
32. 关于 Python 程序中与“缩进”有关的说法中, 以下选项中正确的是
A 缩进统一为 4 个空格
B 缩进可以用在任何语句之后, 表示语句间的包含关系
C 缩进在程序中长度统一且强制使用
D 缩进是非强制性的, 仅为了提高代码可读性
33. 以下选项中可访问字符串 s 从右侧向左第三个字符的是
A s[3] B s[: -3] C s[-3] D s[0:-3]
34. Python3.0 正式发布的年份是
A 1990 B 2018 C 2002 D 2008
35. 以下选项中, 不是 IPO 模型一部分的是
A Program B Input C Output D Process
36. 以下选项中, 不是 Python 语言合法命名的是
A 5MyGod B MyGod5 C _MyGod_ D MyGod
37. 在 Python 函数中, 用于获取用户输入的是
A input() B print() C Eval() D get()
38. 给标识符关联名字的过程是
A 赋值语句 B 命名 C 表达 D 生成语句
39. IDLE 菜单中创建新文件的快捷键是
A Ctrl+] B Ctrl+F C Ctrl+N D Ctrl+[

40. IDLE 菜单中将选中区域缩进的快捷键是
A Ctrl+] B Ctrl+S C Ctrl+A D Ctrl+C
41. IDLE 菜单中将选中区域取消缩进的快捷键是
A Ctrl+O B Alt+C C Ctrl+V D Ctrl+ [
42. IDLE 菜单中将选中区域注释的快捷键是
A Alt+3 B Alt+G C Alt+Z D Alt+4
43. IDLE 菜单中将选中区域取消注释的快捷键是
A Alt+Z B Alt+3 C Alt+G D Alt+4
44. IDLE 菜单将选中区域的空格替换为 Tab 的快捷键是
A Alt+V B Alt+C C Alt+5 D Alt+6
- 45.IDLE 菜单将选中区域的 Tab 替换为空格的快捷键是
A Alt+0 B Alt+5 C Alt+6 D Alt+C
46. 以下选项中，不是 Python 打开方式的是
A Office B Windows 系统的命令行工具
C 带图形界面的 Python Shell—IDLE D 命令行版本的 Python Shell-Python 3.x
- 47.查看 Python 是否安装成功的命令是
A Win + R B PyCharm C python3.4 -v D exit ()
48. 以下选项中,不是 Python IDE 的是
A PyCharm B Jupyter Notebook
C Spyder D R studio
49. Python 为源文件指定系统默认字符编码的声明是
A # coding:cp936 B # coding:GB2312
C # coding:utf—8 D #coding: GBK

50.下面代码的语法错误显示是

```
print "Hello World! "
```

- A SyntaxError: Missing parentheses in call to 'print'
B <built-in function print> <o: p></o:p>
C NameError: name 'raw_print' is not defined
D SyntaxError: invalid character in identifier

二、基本数据类型

1.关于 Python 的数字类型，以下选项中描述错误的是

- A 复数类型虚部为 0 时，表示为 1+0j
B 1.0 是浮点数，不是整数
C 浮点数也有十进制、二进制、八进制和十六进制等表示方式
D 整数类型的数值一定不会出现小数点

2. 下面代码的输出结果是

```
x = 12.34
```

```
print(type(x))
```

- A <class 'float'> B <class 'complex'>
C <class 'bool'> D <class 'int'>

3.下面代码的输出结果是

```
print(pow (2,10))
```

- A 100 B 12 C 1024 D 20

4.下面代码的输出结果是

x=0b1010

print (x)

A 1024 B 10 C 16 D 256

5.下面代码的输出结果是

x=0o1010

print (x)

A 10 B 1024 C 520 D 2768

6.下面代码的输出结果是

x=0x1010

print(x)

A 4112 B 520 C 10 D 1024

7.关于 Python 的浮点数类型，以下选项中描述错误的是

A 浮点数类型与数学中实数的概念一致，表示带有小数的数值

B sys.float_info 可以详细列出 Python 解释器所运行系统的浮点数各项参数

C Python 语言的浮点数可以不带小数部分

D 浮点数有两种表示方法：十进制表示和科学计数法

8.关于 Python 的复数类型，以下选项中描述错误的是

A 复数类型表示数学中的复数

B 对于复数 z，可以用 z.imag 获得实数部分

C 复数的虚数部分通过后缀“J”或“j”来表示

D 对于复数 z，可以用 z.real 获得实数部分

9.下面代码的输出结果是

z = 12. 12 + 34j

print(z. real)

A 34 B 34.0 C 12. 12 D 12

10.下面代码的输出结果是

z = 12.34 + 34j

print (z.imag)

A 12. 12 B 34.0 C 12 D 34

11.下面代码的输出结果是

x=10

y=-1+2j

print(x+y)

A (9+2j) B 11 C 2j D 9

12.下面代码的输出结果是

x=10

y=3

print (x%y,x**y)

A 1 1000 B 3 30 C 3 1000 D 1 30

13.下面代码的输出结果是

x=10

y=4

print(x/y, x//y)

A 2 2. 5 B 2.5 2. 5 C 2.5 2 D 2 2

14. 下面代码的输出结果是

```
x=10
```

```
y=3
```

```
print (divmod (x,y))
```

A 3,1 B (3, 1) C (1,3) D 1,3

15. 下面代码的输出结果是

```
x=3.1415926
```

```
print (round(x,2) ,round (x))
```

A 2 2 B 6.28 3 C 3. 14 3 D 3 3.14

16. 下面代码的输出结果是

```
a = 5
```

```
b = 6
```

```
c = 7
```

```
print(pow(b, 2) -4 * a * c)
```

A 104 B 系统报错 C -104 D 36

17. 关于 Python 字符串，以下选项中描述错误的是

- A 字符串可以保存在变量中，也可以单独存在
- B 字符串是一个字符序列，字符串中的编号叫“索引”
- C 可以使用 datatype()测试字符串的类型
- D 输出带有引号的字符串，可以使用转义字符\

18.下面代码的执行结果是

```
a = 123456789
```

```
b = " * "
```

```
print(" {0: {2}} {1}, } \n{0:{2} ^{1} },} \n{0:{2} <{1},} ".format(a,20, b))
```

A * * * * * 123, 456, 789 B * * * * 123, 456, 789 * * * * *
 * * * * 123, 456, 789 * * * * * * * * * * * * * 123, 456, 789
 123,456, 789 * * * * * 123, 456, 789 * * * * * * * *
C * * * * 123, 456, 789 * * * * * D * * * * * * * * 123,456, 789
 123, 456,789 * * * * * 123, 456,789 * * * * * * * *
 * * * * * * * * 123,456, 789 * * * * 123,456, 789 * * * * *

19.下面代码的执行结果是

```
a = 10.99
```

```
print(complex(a) )
```

A 10.99 B (10.99+0j) C 10.99+0j D 0. 99

20.下面代码的执行结果是

```
>>> x = "Happy Birthday to you! "
```

```
>>> x * 3
```

- A 系统报错
- B Happy Birthday to you!
- C 'Happy Birthday to you! Happy Birthday to you!Happy Birthday to you!'
- D Happy Birthday to you!
Happy Birthday to you!
Happy Birthday to you!

21.关于 Python 字符编码，以下选项中描述错误的是

- A ord(x) 和 chr (x) 是一对函数
- B Python 默认采用 Unicode 字符编码
- C chr (x)将字符转换为 Unicode 编码
- D Python 可以处理任何字符编码文本

22.给出如下代码

```
s = "Alice"
```

```
print (s[::-1] )
```

上述代码的输出结果是

- A ecilA
- B ALICE
- C Alice
- D Alic

23.给出如下代码

```
s= "abcdefghijklmn"
```

```
print (s[1: 10: 3])
```

上述代码的输出结果是

- A behk
- B adgj
- C beh
- D adg

24.给出如下代码

```
for i in range(12) :
```

```
    print(chr(ord("") +i) ,end="")
```

以下选项描述错误的是

- A 输出结果为
- B 系统报错
- C chr (x) 函数返回 Unicode 编码对应的字符
- D ord (")返回" " 字符对应的 Unicode 编码

25.下面代码的输出结果是

```
>>> hex (255)
```

- A '0eff'
- B '0off'
- C '0xff'
- D '0bff'

26.下面代码的输出结果是

```
> >> oct (-255)
```

- A '0d-377'
- B '0o-377'
- C '-0d377'
- D '-0o377'

27。下面代码的输出结果是

```
>>> bin(10)
```

- A '0o1010'
- B '0d1010'
- C '0b1010'
- D '0x1010'

28。给出如下代码 以下选项中描述正确的是

```
for i in range(6):
```

```
    print (chr (ord(9801)+i), end="")
```

- A chr ("a")返回"a"字符对应的 Unicode 编码
- B 系统报错
- C 输出结果为
- D ord(x) 函数返回 x 的 Unicode 编码对应的字符

29。给出如下代码： 如下描述错误的是

```
for i in range (10):
```

```
    print(chr(ord("! ") +i), end=" ")
```

- A 系统报错
- B ord("! ")返回"! "字符对应的 Unicode 编码
- C 输出结果为! " # \$ % & ' () *

D chr(x) 函数返回 Unicode 编码对应的字符

30. 下列选项中输出结果是 True 的是

A >>> isinstance(255,int)

B >>> chr(13).isprintable()

C >>> "Python".islower()

D >>> chr(10).isnumeric()

31. 下面代码的输出结果是

```
s1 = "The python language is a scripting language."
```

```
s1.replace('scripting','general')
```

```
print(s1)
```

A The python language is a scripting language。

B 系统报错

C ['The', 'python', 'language', 'is', 'a', 'scripting', 'language。']

D The python language is a general language。

32. 下面代码的输出结果是

```
s1 = "The python language is a scripting language."
```

```
s2 = s1.replace('scripting', 'general')
```

```
print(s2)
```

A The python language is a scripting language.

B ['The', 'python', 'language', 'is', 'a', 'scripting', 'language。']

C 系统报错

D The python language is a general language。

33. 下面代码的输出结果是

```
s = "The python language is a cross platform language。"
```

```
print(s.find('language',30))
```

A 系统报错

B 40

C 11

D 10

34. 下面代码的输出结果是

```
s = "The python language is a multimodel language。"
```

```
print(s.split(' '))
```

A Thepythonlanguageisamultimodellanguage.

B ['The', 'python', 'language', 'is', 'a', 'multimodel', 'language。']

C The python language is a multimodel language.

D 系统报错

35. 下面代码的输出结果是

```
a="Python"
```

```
b="A Superlanguage"
```

```
print("{}: -> 10}: {:<19}".format(a,b))
```

A ——Python: A Superlanguage——

B ----Python:————A Superlanguage

C The python language is a multimodel language.

D Python-----:---A Superlanguage

36. 以下选项中，输出结果为 False 的是

A >> > 5 is 5

B >>> False != 0

C >>> 5 is not 4

D >>> 5 != 4

37。下面代码的输出结果是

```
>>> True - False
```

A 1 B -1 C True D 0

38.下面代码的输出结果是

```
a = 2
```

```
b = 2
```

```
c = 2.0
```

```
print(a == b, a is b, a is c)
```

A True False False

B True False True

C False False True

D True True False

39.#以下选项中，输出结果为 False 的是

A >>> 'python' < 'pypi'

B > >> 'ABCD' == 'abcd'.upper()

C > >> 'python123' > 'python'

D >>> ' ' < 'a'

40。下面代码的输出结果是

```
>>> a,b, c,d, e,f= 'Python'
```

```
>>> b
```

A 1 B 出错 C 'y' D 0

41.下面代码的输出结果是

```
> >> a = b = c =123
```

```
> >> print (a,b, c)
```

A 0 0 123 B 出错 C 1 1 123 D 123 123 123

42。下面代码的输出结果是

```
>>> True / False
```

A True B -1 C 0 D 系统报错

43。下面代码的输出结果是

```
x = 1
```

```
x *= 3+5**2
```

```
print(x)
```

A 29 B 28 C 13 D 14

44。下面代码的输出结果是

```
a = 5/3+5//3
```

```
print(a)
```

A 5。4 B 2。6666666666666667

C 3。333333 D 14

45。下面代码的输出结果是

```
a = "alex"
```

```
b = a.capitalize ()
```

```
print (a, end=", ")
```

```
print (b)
```

A alex,ALEX B ALEX, alex C alex, Alex D Alex,Alex

46. 下面代码的输出结果是

```
a = 20
b = a | 3
a &= 7
print(b ,end="",")
print (a)
```

A 6.66667, 4 B 4,6.66667 C 4,23 D 23,4

47.下面代码的输出结果是

```
a = "ac"
b = "bd"
c = a + b
print (c)
```

A dbac B abcd C acbd D bdac

48. 下面代码的输出结果是

```
str1 = "mysqlsqlserverPostgresQL"
str2 = "sql"
ncount = str1。count(str2)
print (ncount)
```

A 2 B 5 C 4 D 3

49.下面代码的输出结果是

```
>>> > True / False
```

A True B 1 C 出错 D False

50. 下面代码的输出结果是

```
str1 = "mysqlsqlserverPostgresQL"
str2 = "sql"
ncount = str1.count (str2,10)
print(ncount)
```

A 0 B 3 C 4 D 2

三、程序的控制结构

1. 关于 Python 的分支结构，以下选项中描述错误的是

- A Python 中 if-elif—else 语句描述多分支结构
- B 分支结构使用 if 保留字
- C Python 中 if-else 语句用来形成二分支结构
- D 分支结构可以向已经执行过的语句部分跳转

2. 关于 Python 循环结构，以下选项中描述错误的是

- A break 用来跳出最内层 for 或者 while 循环,脱离该循环后程序从循环代码后继续执行
- B 每个 continue 语句只有能力跳出当前层次的循环
- C 遍历循环中的遍历结构可以是字符串、文件、组合数据类型和 range()函数等
- D Python 通过 for、while 等保留字提供遍历循环和无限循环结构

3. 关于 Python 循环结构，以下选项中描述错误的是

- A continue 结束整个循环过程,不再判断循环的执行条件
- B 遍历循环中的遍历结构可以是字符串、文件、组合数据类型和 range()函数等
- C Python 通过 for、while 等保留字构建循环结构
- D continue 用来结束当前当次语句，但不跳出当前的循环体

4. 下面代码的输出结果是

```
for s in "HelloWorld":  
    if s=="W":  
        continue  
    print (s,end="")
```

- A Hello B HelloWorld C Helloorld D World

5. #下面代码的输出结果是

```
for s in "HelloWorld":  
    if s=="W":  
        break  
    print (s, end="")
```

- A HelloWorld B Helloorld C World D Hello

6. 于程序的异常处理，以下选项中描述错误的是

- A 编程语言中的异常和错误是完全相同的概念
B 程序异常发生后经过妥善处理可以继续执行
C 异常语句可以与 else 和 finally 保留字配合使用
D Python 通过 try、except 等保留字提供异常处理功能

7. 关于 Python 遍历循环，以下选项中描述错误的是

- A 遍历循环通过 for 实现
B 无限循环无法实现遍历循环的功能
C 遍历循环可以理解为从遍历结构中逐一提取元素，放在循环变量中,对于所提取的每个元素只执行一次语句块
D 遍历循环中的遍历结构可以是字符串、文件、组合数据类型和 range()函数等

8. 关于 Python 的无限循环,以下选项中描述错误的是

- A 无限循环一直保持循环操作,直到循环条件不满足才结束
B 无限循环也称为条件循环
C 无限循环通过 while 保留字构建
D 无限循环需要提前确定循环次数

9. 下面代码的输出结果是

```
for i in "Python":  
    print (i,end=" ")
```

- A P,y, t, h,o,n, B P y t h o n C Python D P y t h o n

10. 给出如下代码:

```
import random  
num = random.randint (1, 10)  
while True:  
    guess = input ()  
    i = int (guess)  
    if i == num:  
        print ("你猜对了")  
        break  
    elif i < num:  
        print(" 小了" )  
    elif i > num:
```

```
print("大了")
```

以下选项中描述错误的是

- A random。randint(1,10)生成[1， 10] 之间的整数
- B “import random”这行代码是可以省略的
- C 这段代码实现了简单的猜数字游戏
- D “while True:” 创建了一个永远执行的 While 循环

11. 给出如下代码:

```
a=3
```

```
while a > 0:
```

```
    a -= 1
```

```
    print (a,end=" ")
```

以下选项中描述错误的是:

- A a -= 1 可由 a = a - 1 实现
- B 条件 a > 0 如果修改为 a < 0 程序执行会进入死循环
- C 使用 while 保留字可创建无限循环
- D 这段代码的输出内容为 2 1 0

12. 下列快捷键中能够中断 (Interrupt Execution) Python 程序运行的是

- A F6
- B Ctrl + Q
- C Ctrl + C
- D Ctrl + F6

13. 给出如下代码:

```
sum = 0
```

```
for i in range(1, 11):
```

```
    sum += i
```

```
    print(sum)
```

以下选项中描述正确的是:

- A 循环内语句块执行了 11 次
- B sum += i 可以写为 sum + = i
- C 如果 print (sum) 语句完全左对齐, 输出结果不变
- D 输出的最后一个数字是 55

14. 关于 break 语句与 continue 语句的说法中,以下选项中不正确的是

- A continue 语句类似于 break 语句, 也必须在 for、while 循环中使用
- B break 语句结束循环, 继续执行循环语句的后续语句
- C 当多个循环语句嵌套时, break 语句只适用于最里层的语句
- D continue 语句结束循环, 继续执行循环语句的后续语句

15. random。uniform(a,b) 的作用是

- A 生成一个 [a, b]之间的随机整数
- B 生成一个(a, b) 之间的随机数
- C 生成一个均值为 a,方差为 b 的正态分布
- D 生成一个 [a, b] 之间的随机小数

16. 实现多路分支的最佳控制结构是

- A if
- B try
- C if-elif—else
- D if—else

17. 给出下面代码:

```
age=23
```

```
start=2
```

```
if age%2 !=0:
```

```
start=1
for x in range(start,age+2, 2):
    print(x)
```

上述程序输出值的个数是：

- A 10 B 12 C 16 D 14

18. 下面代码的执行结果是

```
print(pow(3, 0.5)*pow (3, 0.5)==3)
```

- A True B pow(3, 0.5) * pow(3, 0.5)==3
C False D 3

19. 给出下面代码：

```
k=10000
while k>1:
    print(k)
    k=k/2
```

上述程序的运行次数是：

- A 14 B 1000 C 15 D 13

20. 关于 Python 语句 P= - P，以下选项中描述正确的是

- A P 的绝对值 B 给 P 赋值为它的负数
C P=0 D P 等于它的负数

21. 以下选项中能够实现 Python 循环结构的是

- A loop B do... for C while D if

22. 用来判断当前 Python 语句在分支结构中的是

- A 引号 B 冒号 C 大括号 D 缩进

23. 以下选项中描述正确的是

- A 条件 24<=28 <25 是合法的,且输出为 False
B 条件 35 <=45<75 是合法的,且输出为 False
C 条件 24<=28<25 是不合法的
D 条件 24<=28<25 是合法的，且输出为 True

24. 于 while 保留字，以下选项中描述正确的是

- A while True: 构成死循环，程序要禁止使用
B 使用 while 必须提供循环次数
C 所有 while 循环功能都可以用 for 循环替代
D 使用 while 能够实现循环计数

25. random 库中用于生成随机小数的函数是

- A randrange() B random() C randint() D getrandbits()

26. 以下选项中能够最简单地在列表['apple', 'pear', 'peach', 'orange'] 中随机选取一个元素的是

- A sample () B random() C choice () D shuffle ()

27. Python 异常处理中不会用到的关键字是

- A finally B else C try D if

28. 下面代码的输出结果是

```
for i in range(1,6):
    if i%3 == 0:
        break
```

```
else:  
    print (i, end =", ")
```

A 1,2, 3, B 1,2,3,4,5, 6 C 1, 2, D 1, 2, 3,4,5,

29. 下面代码的输出结果是

```
for i in range(1, 6):
```

```
    if i/3 == 0:
```

```
        break
```

```
    else:
```

```
        print(i, end =", ")
```

A 1,2, 3, B 1, 2,3,4,5, C 1,2,3, 4, D 1,2,

30. 下面代码的输出结果是

```
sum = 0
```

```
for i in range (2, 101):
```

```
    if i % 2 == 0:
```

```
        sum += i
```

```
    else:
```

```
        sum -= i
```

```
print (sum)
```

A —50 B 51 C 50 D 49

31. 下面代码的输出结果是

```
sum=0
```

```
for i in range(0, 100):
```

```
    if i%2==0:
```

```
        sum—=i
```

```
    else:
```

```
        sum+=i
```

```
print(sum)
```

A -50 B 49 C 50 D -49

32. 下面代码的输出结果是

```
for i in range(1,10, 2) :
```

```
    print(i, end=",")
```

A 1, 4,

B 1, 4,7,

C 1, 3,5, 7, 9,

D 1,3,

33. 下面代码的输出结果是

```
sum = 1
```

```
for i in range (1,101) :
```

```
    sum += i
```

```
print(sum)
```

A 5052 B 5051 C 5049 D 5050

34. 下面代码的输出结果是

```
a = []
```

```
for i in range (2,10) :
```

```

count = 0
for x in range(2,i-1) :
    if i % x == 0:
        count += 1
if count != 0:
    a.append (i)
print (a)

```

A [3 , 5,7 , 9] B [4, 6, 8, 9] C [4,6,8,9 , 10] D [2 , 3,5,7]

35. 下面代码的输出结果是

```

x2 = 1
for day in range(4, 0, -1):
    x1 = (x2 + 1) * 2
    x2 = x1
print(x1)

```

A 46
B 23
C 94
D 190

36. 下面代码的输出结果是

```

for num in range(2,10):
    if num > 1:
        for i in range (2, num):
            if (num % i) == 0:
                break
        else:
            print(num)

```

A 4, 6, 8, 9 B 2,4,6,8,10 C 2,4,6, 8 D 2, 3, 5, 7,

37. 下面代码的输出结果是

```

for n in range (100, 200):
    i = n // 100
    j = n // 10 % 10
    k = n % 10
    if n == i ** 3 + j ** 3 + k ** 3:
        print(n)

```

A 159 B 157 C 152 D 153

38. 下面代码的输出结果是

```

a = 2.0
b = 1.0
s = 0
for n in range (1, 4):
    s += a / b
    t = a

```

```
a = a + b
b = t
print (round(s, 2))
```

A 5.17 B 8.39 C 3.5 D 6.77

39. 下面代码的输出结果是

```
for a in ["torch","soap","bath"] :
```

```
    print (a)
```

A torch
soap
bath

B torch,soap,bath

C torch soap bath

D torch,soap, bath,

40. 下面代码的输出结果是

```
for a in 'mirror':
```

```
    print(a, end="")
    if a == 'r':
        break
```

A mir B mirror C mi D mirror

41. 下面代码的输出结果是

```
s = 0
```

```
while (s <=1):
    print(' 计数: ',s)
    s = s + 1
```

A 计数:1 B 计数: 0 计数: 1 C 计数:0

D 出错

42. 下面代码的输出结果是

```
s = 1
```

```
while (s <=1):
    print ('计数: ',s)
    s = s + 1
```

A 计数: 0 B 出错

计数: 1

C 计数: 0 D 计数:1

43. 下面代码的输出结果是

```
for i in ["pop star"]:
```

```
    pass
    print(i, end = "")
```

A 无输出 B pop star C 出错 D popstar

44. 给出下面代码:

```
i = 1
while i < 6:
    j = 0
    while j < i:
```

```

        print ("*",end='')
        j += 1
    print("\n")
    i += 1

```

以下选项中描述错误的是：

- A 第 i 行有 i 个星号*
- B 输出 5 行
- C 执行代码出错
- D 内层循环 j 用于控制每行打印的 * 的个数

45. 给出下面代码：

```

for i in range(1, 10):
    for j in range(1,i+1):
        print("{} * {} = {}".format(j,i*i*j) ,end = ")
    print("")

```

以下选项中描述错误的是：

- A 内层循环 i 用于控制一共打印 9 列
- B 也可使用 While 嵌套循环实现打印九九乘法表
- C 执行代码,输出九九乘法表
- D 执行代码出错

46. 下面代码的输出结果是

```
a = 1.0
```

```

if isinstance (a,int):
    print("{} is int".format(a))
else:
    print(" {} is not int".format (a))

```

- A 出错
- B 1.0 is int
- C 无输出
- D 1.0 is not int

47. 下面代码的输出结果是

```

a = {}
if isinstance(a,list) :
    print(" {} is list".format (a) )
else:
    print ("{} is {}".format("a",type(a)))

```

- A a is list
- B 出错
- C 无输出
- D a is <class 'dict' >

48. 下面代码的输出结果是

```

a = [1,2,3]
if isinstance (a,float) :
    print(" {} is float".format(a) )
else:
    print(" {} is not float".format(a))

```

- A a is float
- B a is <class ' float t' >
- C [1, 2, 3] is not float
- D 出错

49. 给出下面代码：

```

a = input ("").split (" , ")
if isinstance(a,list) :
    print ("{} is list".format(a))
else:

```

```
print("{} is not list".format(a))
```

代码执行时,从键盘获得 1,2,3, 则代码的输出结果是:

- A 执行代码出错 B 1, 2,3 is not list
C ['1','2','3'] is list D 1,2,3 is list

50. 给出下面代码:

```
a = input ("").split(", ")
```

```
x = 0
```

```
while x < len(a):
```

```
    print (a[x], end="")
```

```
    x += 1
```

代码执行时, 从键盘获得 a, b,c,d,则代码的输出结果是:

- A 执行代码出错 B abcd C 无输出 D a,b,c,d

四、函数和代码复用

1. 关于递归函数的描述, 以下选项中正确的是

- A 函数名称作为返回值
B 包含一个循环结构
C 函数比较复杂
D 函数内部包含对本函数的再次调用

2. 关于递归函数基例的说明,以下选项中错误的是

- A 递归函数必须有基例
B 递归函数的基例不再进行递归
C 每个递归函数都只能有一个基例
D 递归函数的基例决定递归的深度

3. 以下选项中, 不属于函数的作用的是

- A 提高代码执行速度
B 增强代码可读性
C 降低编程复杂度
D 复用代码

4. 假设函数中不包括 global 保留字, 对于改变参数值的方法, 以下选项中错误的是

- A 参数是列表类型时, 改变原参数的值
B 参数是组合类型(可变对象)时, 改变原参数的值
C 参数的值是否改变与函数中对变量的操作有关, 与参数类型无关
D 参数是整数类型时, 不改变原参数的值

5 在 Python 中, 关于函数的描述, 以下选项中正确的是。

- A 函数 eval()可以用于数值表达式求值, 例如 eval(" 2*3+1")
B Python 函数定义中没有对参数指定类型, 这说明, 参数在函数中可以当作任意类型使用
C 一个函数中只允许有一条 return 语句
D Python 中,def 和 return 是函数必须使用的保留字

6. 给出如下代码:

```
def func (a, b):
```

```
    c=a**2+b
```

```
    b=a
```

```
    return c
```

a=10

b=100

c=func(a,b) +a

以下选项中描述错误的是

- A 执行该函数后，变量 a 的值为 10
- B 执行该函数后，变量 b 的值为 100
- C 执行该函数后，变量 c 的值为 200
- D 该函数名称为 func

7. 在 Python 中，关于全局变量和局部变量，以下选项中描述不正确的是

- A 一个程序中的变量包含两类：全局变量和局部变量
- B 全局变量不能和局部变量重名
- C 全局变量在程序执行的全过程有效
- D 全局变量一般没有缩进

8. 关于面向对象和面向过程编程描述，以下选项中正确的是

- A 面向对象编程比面向过程编程更为高级
- B 所有面向对象编程能实现的功能采用面向过程同样能完成
- C 面向对象和面向过程是编程语言的分类依据
- D 模块化设计就是面向对象的设计

9. 以下选项中，对于递归程序的描述错误的是

- A 书写简单
- B 执行效率高
- C 递归程序都可以有非递归编写方法
- D 一定要有基例

10. 下面代码的输出结果是

```
>>> f=lambda x, y: y+x
```

```
>>> f(10, 10)
```

- A 100 B 10 C 20 D 10,10

11. 关于形参和实参的描述，以下选项中正确的是

- A 参数列表中给出要传入函数内部的参数，这类参数称为形式参数，简称形参
- B 程序在调用时，将形参复制给函数的实参
- C 函数定义中参数列表里面的参数是实际参数，简称实参
- D 程序在调用时，将实参复制给函数的形参

12. 关于 lambda 函数，以下选项中描述错误的是

- A lambda 不是 Python 的保留字
- B 定义了一种特殊的函数
- C lambda 函数也称为匿名函数
- D lambda 函数将函数名作为函数结果返回

13 以下选项中，对于函数的定义错误的是

- A def vfunc(a,b=2): B def vfunc(* a,b) :
- C def vfunc(a, b): D def vfunc(a,* b):

14. 关于函数的参数，以下选项中描述错误的是

- A 在定义函数时，如果有些参数存在默认值，可以在定义函数时直接为这些参数指定默认值
- B 在定义函数时，可以设计可变数量参数，通过在参数前增加星号 (*) 实现

- C 可选参数可以定义在非可选参数的前面
D 一个元组可以传递给带有星号的可变参数
15. 关于 `return` 语句, 以下选项中描述正确的是
- A 函数必须有一个 `return` 语句
B 函数中最多只有一个 `return` 语句
C `return` 只能返回一个值
D 函数可以没有 `return` 语句
16. 关于函数, 以下选项中描述错误的是
- A 函数是一段具有特定功能的、可重用的语句组
B Python 使用 `def` 保留字定义一个函数
C 函数能完成特定的功能, 对函数的使用不需要了解函数内部实现原理, 只要了解函数的输入输出方式即可。
D 使用函数的主要目的是降低编程难度和代码重用
17. 关于 Python 的全局变量和局部变量, 以下选项中描述错误的是
- A 使用 `global` 保留字声明简单数据类型变量后, 该变量作为全局变量使用
B 简单数据类型变量无论是否与全局变量重名, 仅在函数内部创建和使用, 函数退出后变量被释放
C 全局变量指在函数之外定义的变量, 一般没有缩进, 在程序执行全过程有效
D 局部变量指在函数内部使用的变量, 当函数退出时, 变量依然存在, 下次函数调用可以继续使用
18. 关于 Python 的 `lambda` 函数, 以下选项中描述错误的是
- A `f = lambda x, y: x+y` 执行后, `f` 的类型为数字类型
B `lambda` 用于定义简单的、能够在一行内表示的函数
C 可以使用 `lambda` 函数定义列表的排序原则
D `lambda` 函数将函数名作为函数结果返回
19. 下面代码实现的功能描述为

```
def fact(n):  
    if n==0:  
        return 1  
    else:  
        return n * fact(n-1)  
num = eval (input ("请输入一个整数: "))  
print(fact (abs(int(num))))
```

- A 接受用户输入的整数 `N`, 输出 `N` 的阶乘值
B 接受用户输入的整数 `N`, 判断 `N` 是否是素数并输出结论
C 接受用户输入的整数 `N`, 判断 `N` 是否是水仙花数
D 接受用户输入的整数 `N`, 判断 `N` 是否是完数并输出结论

20. 给出如下代码:

```
def fact (n):  
    s = 1  
    for i in range (1, n+1) :  
        s *= i  
    return s
```

以下选项中描述错误的是

A fact(n) 函数功能为求 n 的阶乘

C s 是局部变量

B range() 函数是 Python 内置函数

D 代码中 n 是可选参数

21. 给出如下代码:

```
ls = ["car", "truck"]
```

```
def funC (a):
```

```
    ls.append (a)
```

```
    return
```

```
funC("bus")
```

```
print(ls)
```

以下选项中描述错误的是

A funC (a) 中的 a 为非可选参数

B ls.append(a) 代码中的 ls 是全局变量

C ls.append(a) 代码中的 ls 是列表类型
'truck']

D 执行代码输出结果为[' car',

22. 给出如下代码:

```
ls = ["car", "truck"]
```

```
def funC (a):
```

```
    ls=[]
```

```
    ls.append (a)
```

```
    return
```

```
funC ("bus")
```

```
print (ls)
```

以下选项中描述错误的是

A 执行代码输出结果为 [' car', 'truck' , ' bus']

B 代码函数定义中, ls.append(a)中的 ls 是局部变量

C 执行代码输出结果为['car' , ' truck']

D ls.append (a) 代码中的 ls 是列表类型

23. 给出如下代码:

```
import turtle
```

```
def drawLine(draw):
```

```
    turtle.pendown() if draw else turtle.penup()
```

```
    turtle.fd (50)
```

```
    turtle.right (90)
```

```
drawLine(True)
```

```
drawLine (True)
```

```
drawLine(True)
```

```
drawLine(True)
```

以下选项中描述错误的是

A 代码 drawLine (True) 中 True 替换为 - 1, 运行代码结果不变

B 代码 drawLine (True) 中 True 替换为 0, 运行代码结果不变

C 代码 def drawLine(draw) 中的 draw 可取值 True 或者 False

D 运行代码, 在 Python Turtle Graphics 中, 绘制一个正方形

24. 给出如下代码:

```
import turtle
```

```
def drawLine (draw):
```

```

        turtle.pendown () if draw else turtle.penup ()
        turtle.fd(50)
        turtle.right(90)
drawLine(True)
drawLine (0)
drawLine(True)
drawLine (True)
turtle.left (90)
drawLine (0)
drawLine (True)
drawLine(True)

```

以下选项中描述错误的是：

- A 代码 drawLine(True)中 True 替换为 - 1，运行代码结果不变
- B 代码 drawLine(True)中 True 替换为 0,运行代码结果不变
- C 代码 def drawLine (draw) 中的 draw 可取数值 0、1、 - 1 等
- D 运行代码，在 Python Turtle Graphics 中,绘制一个数码管数字 2

25. 下面代码的运行结果是

```

def func(num):
    num += 1

```

```

a=10
func(a)
print (a)

```

- A 11
- B int
- C 10
- D 出错

26. 下面代码的输出结果是

```

def func (a,b):
    return a>> b

```

```

s = func (5,2)
print (s)

```

- A 20
- B 1
- C 6
- D 12

27. 下面代码的输出结果是

```

def func(a, b):
    a *= b
    return a

```

```

s = func (5, 2)
print (s)

```

- A 20
- B 1
- C 12
- D 10

28. 下面代码的输出结果是

```

def f2(a) :
    if a > 33:
        return True

```

```

li = [11, 22, 33, 44, 55]
res = filter (f2, li)
print(list (res) )

```

- A [44, 55]
- B [11, 33, 55]
- C [22, 33,44]
- D [33, 44,55]

29. 下面代码的输出结果是

```
def fib(n):  
    a,b = 1, 1  
    for i in range(n-1):  
        a, b = b,a+b  
    return a  
print (fib (7))
```

A 5 B 13 C 21 D 8

30. 下面代码的输出结果是

```
def hello_world():  
    print('ST', end="*")  
def three_hellos():  
    for i in range(3):  
        hello_world ()  
three_hellos()
```

A ST*ST*ST* B ST* C ST*ST* D ***

31. 下面代码的输出结果是

```
def exchange(a, b):  
    a,b = b,a  
    return (a,b)  
x = 10  
y = 20  
x, y = exchange(x,y)  
print (x,y)
```

A 20 10 B 20 20 C 20, 10 D 10 10

32. 下面代码的输出结果是

```
MA = lambda x, y : (x > y) * x + (x < y) * y  
MI = lambda x,y : (x > y) * y + (x < y) * x  
a = 10  
b = 20  
print (MA(a, b))  
print (MI (a, b) )
```

33. 关于下面的代码，以下选项中描述正确的是

```
>>> list(range(0,10, 2))
```

A 按可变参数调用 B 按关键字参数调用
C 执行结果为 0, 2, 4, 6, 8 D 按位置参数调用

34. 关于下面代码,以下选项中描述正确的是

```
def fact(n, m=1) :  
    s = 1  
    for i in range (1, n+1) :  
        s *= i
```

```
return s//m
```

```
print (fact(m=5, n=10))
```

A 参数按照名称传递

B 按位置参数调用

C 执行结果为 10886400

D 按可变参数调用

35. 关于函数的返回值，以下选项中描述错误的是

A 函数可以返回 0 个或多个结果

B 函数必须有返回值

C 函数可以有 return，也可以没有

D return 可以传递 0 个返回值，也可以传递任意多个返回值

36. 关于函数局部变量和全局变量的使用规则,以下选项中描述错误的是

A 对于组合数据类型的变量,如果局部变量未真实创建，则是全局变量

B 对于基本数据类型的变量，无论是否重名，局部变量与全局变量不同

C 可以通过 global 保留字在函数内部声明全局变量

D return 不可以传递任意多个函数局部变量返回值

37. 关于函数，以下选项中描述错误的是

A 函数使用时需要了解函数内部实现细节

B 函数主要通过接口(interface)与外界通信,传递信息

C 函数:具有特定功能的可重用代码片段，实现解决某个特定问题的算法

D 函数在需要时被调用,其代码被执行

38. 关于函数的目的与意义，以下选项中描述错误的是

A 程序功能抽象,以支持代码重用

B 函数能调用未实现的函数

C 有助于采用分而治之的策略编写大型复杂程序

D 使用时无须了解函数内部实现细节

39. 关于函数，以下选项中描述错误的是

A 函数也是数据

B 函数定义语句可执行

C 一条函数定义定义一个用户自定义函数对象

D 函数名称不可赋给其他变量

40. 关于函数的参数传递(parameter passing)，以下选项中描述错误的是

A 实际参数是函数调用时提供的参数

B 函数调用时,需要将形式参数传递给实际参数

C Python 参数传递时不构造新数据对象,而是让形式参数和实际参数共享同一对象

D 形式参数是函数定义时提供的参数

41. 关于函数的关键字参数使用限制，以下选项中描述错误的是

A 关键字参数顺序无限制

B 关键字参数必须位于位置参数之前

C 关键字参数必须位于位置参数之后

D 不得重复提供实际参数

42. 下面代码的输出结果是

```
a = 4
```

```
a ^= 3
```

```
b = a ^ 2
```

```
print(a,end=",")
```

print (b)

A 4,3 B 64, 4096 C 5,7 D 7, 5

43. 执行下面代码,错误的是

def f(x, y = 0, z = 0): pass # 空语句, 定义空函数体

A f(1, , 3) B f (1, 2) C f(1) D f (1, 2, 3)

44. 执行下面代码,错误的是

def f(x, y = 0, z = 0) : pass # 空语句, 定义空函数体

A f (1, z = 3) B f (1, x = 1, z = 3)
C f(1, y = 2, z = 3) D f(z = 3, x = 1, y = 2)

45. 执行下面的代码, 以下选项中正确的是

def f(x, y = 0, z = 0) : pass # 空语句, 定义空函数体

A f (x = 1, 2) B f (1, y = 2, t = 3)
C f (1, x = 1, z = 3) D f (x = 1, y = 2, z = 3)

46. 关于嵌套函数,以下选项中描述错误的是

A 嵌套函数是在函数内部定义函数
B 内层函数仅供外层函数调用, 外层函数之外不得调用
C def f() :

print ("Outer function f")

def g () :

print ("Inner function g")

g()

f()

D def f ():

print("Outer function f")

def g():

print ("Inner function g")

g()

f.g()

47. 下面代码的执行结果是

>>> def area (r, pi = 3.14159):

return pi * r * r

> > area (pi = 3.14, r = 4)

A 出错 B 50.24 C 39.4384 D 无输出

48.下面代码的执行结果是

>>> def area(r, pi = 3.14159):

return pi * r * r

>>> area (3.14, 4)

A 出错 B 39.4384 C 50.24 D 无输出

49. 下面代码的执行结果是

def greeting (args1, *tupleArgs, **dictArgs):

print (args1)

print(tupleArgs)

print (dictArgs)

names = ['HTY', 'LFF', 'ZH']

```
info = {'schoolName': 'NJRU', 'City': 'Nanjing'}
```

```
greeting ('Hello,', *names, **info)
```

A Hello,

('HTY', 'LFF', 'ZH')

```
{'schoolName': 'NJRU', 'City': 'Nanjing'}
```

B ['HTY', 'LFF', 'ZH']

C 出错

D 无输出

50. 下面代码的执行结果是

```
def greeting(args1, *tupleArgs, **dictArgs):
```

```
    print (args1)
```

```
    print(tupleArgs)
```

```
    print(dictArgs)
```

```
names = ['HTY', 'LFF', 'ZH']
```

```
info = {'schoolName': 'NJRU', 'City': 'Nanjing'}
```

```
greeting(*names, 'Hello,', **info)
```

A HTY

B 出错

('LFF', 'ZH', 'Hello,')

```
{'schoolName': 'NJRU', 'City': 'Nanjing'}
```

C 无输出

D ['HTY', 'LFF', 'ZH']

五、组合数据类型

1. 字典 d= {'abc':123, 'def':456, 'ghi': 789}, len(d)的结果是

A 9

B 12

C 3

D 6

2. 关于 Python 的元组类型，以下选项中描述错误的是

A 元组中元素不可以是不同类型

B 元组一旦创建就不能被修改

C Python 中元组采用逗号和圆括号（可选）来表示

D 一个元组可以作为另一个元组的元素，可以采用多级索引获取信息

3. S 和 T 是两个集合，对 S&T 的描述正确的是

A S 和 T 的并运算,包括在集合 S 和 T 中的所有元素

B S 和 T 的差运算，包括在集合 S 但不在 T 中的元素

C S 和 T 的补运算，包括集合 S 和 T 中的非相同元素

D S 和 T 的交运算,包括同时在集合 S 和 T 中的元素

4. S 和 T 是两个集合，对 S\T 的描述正确的是

A S 和 T 的差运算，包括在集合 S 但不在 T 中的元素

B S 和 T 的并运算，包括在集合 S 和 T 中的所有元素

C S 和 T 的交运算，包括同时在集合 S 和 T 中的元素

D S 和 T 的补运算,包括集合 S 和 T 中的非相同元素

5.以下选项中，不是具体的 Python 序列类型的是

A 元组类型

B 字符串类型

C 数组类型

D 列表类型

6. 对于序列 s，能够返回序列 s 中第 i 到 j 以 k 为步长的元素子序列的表达是

A s[i, j, k]

B s (i, j, k)

C s [i; j; k]

D s[i:j: k]

7.设序列 s，以下选项中对 max (s)的描述正确的是

A 返回序列 s 的最大元素,如果有多个相同,则返回一个元组类型

B 返回序列 s 的最大元素,如果有多个相同,则返回一个列表类型

C 返回序列 s 的最大元素,但要求 s 中元素之间可比较

D 一定能够返回序列 s 的最大元素

8.元组变量 t=("cat", "dog", "tiger", "human"), t[::-1]的结果是

A {'human', 'tiger', 'dog', 'cat'}

B ['human', 'tiger', 'dog', 'cat']

C 运行出错

D ('human', 'tiger', 'dog', 'cat')

9.以下选项中不能生成一个空字典的是

A dict()

B { [] }

C { }

D dict([])

10.给定字典 d,以下选项中对 d.keys()的描述正确的是

A 返回一个集合类型,包括字典 d 中所有键

B 返回一种 dict_keys 类型,包括字典 d 中所有键

C 返回一个元组类型,包括字典 d 中所有键

D 返回一个列表类型,包括字典 d 中所有键

11.给定字典 d,以下选项中对 d.values()的描述正确的是

A 返回一个集合类型,包括字典 d 中所有值

B 返回一种 dict_values 类型,包括字典 d 中所有值

C 返回一个元组类型,包括字典 d 中所有值

D 返回一个列表类型,包括字典 d 中所有值

12.给定字典 d,以下选项中对 d.items()的描述正确的是

A 返回一个集合类型,每个元素是一个二元元组,包括字典 d 中所有键值对

B 返回一个列表类型,每个元素是一个二元元组,包括字典 d 中所有键值对

C 返回一个元组类型,每个元素是一个二元元组,包括字典 d 中所有键值对

D 返回一种 dict_items 类型,包括字典 d 中所有键值对

13.给定字典 d,以下选项中对 d.get(x, y)的描述正确的是

A 返回字典 d 中键为 y 的值,如果不存在,则返回 y

B 返回字典 d 中值为 y 的值,如果不存在,则返回 x

C 返回字典 d 中键值对为 x:y 的值

D 返回字典 d 中键为 x 的值,如果不存在,则返回 y

14.给定字典 d,以下选项中对 x in d 的描述正确的是

A 判断 x 是否是在字典 d 中以键或值方式存在

B 判断 x 是否是字典 d 中的值

C x 是一个二元元组,判断 x 是否是字典 d 中的键值对

D 判断 x 是否是字典 d 中的键

15.给定字典 d,以下选项中可以清空该字典并保留变量的是

A d.remove()

B d.clear()

C d.pop()

D del d

16.关于 Python 组合数据类型,以下选项中描述错误的是

A *Python 组合数据类型能够将多个同类型或不同类型的数据组织起来,通过单一的表达使数据操作更有序、更容易

B 序列类型是二维元素向量,元素之间存在先后关系,通过序号访问

C 组合数据类型可以分为 3 类:序列类型、集合类型和映射类型

D *Python 的 str、tuple 和 list 类型都属于序列类型

17.关于 Python 的元组类型,以下选项中描述错误的是

A 元组一旦创建就不能被修改

B 元组中元素不可以是不同类型

C 一个元组可以作为另一个元组的元素，可以采用多级索引获取信息

D *Python 中元组采用逗号和圆括号（可选）来表示

18.关于 Python 的列表，以下选项中描述错误的是

A Python 列表是一个可以修改数据项的序列类型

B Python 列表的长度不可变

C Python 列表用中括号[]表示

D Python 列表是包含 0 个或者多个对象引用的有序序列

19.关于 Python 序列类型的通用操作符和函数,以下选项中描述错误的是

A 如果 s 是一个序列,s=[1, "kate",True], s[3] 返回 True

B 如果 s 是一个序列, x 是 s 的元素, x in s 返回 True

C 如果 s 是一个序列, s=[1," kate", True], s [- 1] 返回 True

D 如果 s 是一个序列,x 不是 s 的元素, x not in s 返回 True

20.下面代码的输出结果是

```
s = ["seashell","gold", "pink", "brown","purple","tomato"]
```

```
print(s [1:4: 2])
```

A ['gold', 'pink', 'brown', 'purple', 'tomato']

B ['gold', 'pink']

C ['gold', 'brown']

D [' gold', ' pink', 'brown']

21.下面代码的输出结果是

```
s = ["seashell", "gold","pink", "brown","purple", "tomato"]
```

```
print (s[4:])
```

A ['purple', 'tomato']

B ['seashell', 'gold', 'pink', 'brown']

C ['purple']

D ['gold' , 'pink',

'brown', 'purple', 'tomato']

22。下面代码的输出结果是

```
s = ["seashell","gold","pink", "brown", "purple","tomato"]
```

```
print (len (s), min (s) ,max(s) )
```

A 6 seashell gold

B 5 pink brown

C 6 brown tomato

D 5 purple tomato

23。给出如下代码：

```
s = list (" 巴老爷有八十八棵芭蕉树,来了八十八个把式要在巴老爷八十八棵芭蕉树下\
住。老爷拔了八十八棵芭蕉树，不让八十八个把式在八十八棵芭蕉树下住。八十八个\
把式烧了八十八棵芭蕉树,巴老爷在八十八棵树边哭。")
```

以下选项中能输出字符“八”出现次数的是

A print (s。count (" 八"))

B print (s。index(" 八"))

C print(s.index(" 八"), 6, len (s))

D print (s。index("八"),6)

24。给出如下代码：

```
s = list ("巴老爷有八十八棵芭蕉树， 来了八十八个把式要在巴老爷八十八棵芭蕉树下住。
老爷拔了八十八棵芭蕉树，不让八十八个把式在八十八棵芭蕉树下住。八十八个把式烧了八
十八棵芭蕉树，巴老爷在八十八棵树边哭。")
```

以下选项中能输出字符“八”第一次出现的索引位置的是

A print(s。index("八"), 6,len(s))

B print (s.count ("八"))

C print(s.index ("八"))

D print(s.index(" 八"),6)

25。下面代码的输出结果是

```
vlist = list(range (5))
```

```
print(vlist)
```

A [0, 1, 2, 3, 4]

B 0, 1, 2, 3, 4,

C 0 1 2 3 4

D 0; 1; 2; 3; 4;

26. 下面代码的输出结果是

```
vlist = list (range (5))
```

```
for e in vlist:
```

```
    print (e, end=",")
```

A 0,1, 2,3, 4,

B [0, 1, 2, 3, 4]

C 0 1 2 3 4

D 0;1; 2; 3; 4;

27.关于 Python 字典，以下选项中描述错误的是

A Python 字典是包含 0 个或多个键值对的集合,没有长度限制，可以根据”键”索引”值”的内容

B 如果想保持一个集合中元素的顺序，可以使用字典类型

C Python 通过字典实现映射

D 字典中对某个键值的修改可以通过中括号[] 的访问和赋值实现

28.给出如下代码：

```
DictColor = { "seashell": "海贝色", "gold": "金色", "pink": "粉红色", "brown": "棕色", "purple": "紫色", "tomato": "西红柿色" }
```

以下选项中能输出“海贝色”的是

A print (DictColor["seashell"])

B print (DictColor。keys ())

C print(DictColor。values())

D print (DictColor [" 海贝色"])

29. 给出如下代码：

```
import random as ran
```

```
listV = []
```

```
ran.seed(100)
```

```
for i in range(10) :
```

```
    i = ran。randint (100, 999)
```

```
    listV。append (i)
```

以下选项中能输出随机列表元素最大值的是

A print (listV.max ())

B print(listV。reverse(i))

C print (listV。pop(i))

D print (max (listV))

30. 给出如下代码:

```
MonthandFlower= { "1 月": "梅花", "2 月": "杏花", "3 月": "桃花", "4 月": "牡丹花", \
"5 月": "石榴花", "6 月": "莲花", "7 月": "玉簪花", "8 月": "桂花", "9 月": "菊花", \
"10 月": "芙蓉花", "11 月": "山茶花", "12 月": "水仙花" }
```

```
n = input(" 请输入 1—12 的月份: ")
```

```
print (n + "月份之代表花:" + MonthandFlower.get(str(n)+ " 月" ))
```

以下选项中描述正确的是

A *MonthandFlower 是一个集合

B *MonthandFlower 是一个元组

C 代码实现了从键盘上获取一个整数(1—12) 来表示月份,输出该月份对应的代表花名

D *MonthandFlower 是一个列表

31. 下面代码的输出结果是

```
list1 = []
```

```
for i in range (1, 11):
    list1.append(i**2)
```

```
print(list1)
```

A [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

B [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

C 错误

D -----Python: -----A Superlanguage

32.下面代码的输出结果是

```
list1 = [i*2 for i in 'Python']
```

```
print (list1)
```

A 错误

B [2, 4, 6, 8, 10, 12]

C Python Python

D ['PP', 'yy', 'tt', 'hh', 'oo', 'nn']

33.下面代码的输出结果是

```
list1 = [m+n for m in 'AB' for n in 'CD']
```

```
print (list1)
```

A ABCD

B AABBBCCDD

C ['AC', 'AD', 'BC', 'BD']

D 错误

34.下面代码的输出结果是

```
list1 = [(m,n) for m in 'AB' for n in 'CD']
```

```
print(list1)
```

A [('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D')]

B ['A', 'B', 'C', 'D']

C * ['AC', 'AD', 'BC', 'BD']

D 错误

35.下面代码的输出结果是

```
list1 = [(m, n) for m in 'ABC' for n in 'ABC' if m != n]
```

```
print(list1)
```

A * [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]

B *['AC', 'AD', 'BC', 'BD']

C 错误

D * [('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D')]

36.下面代码的输出结果是

```
d = {'a':1, 'b': 2, 'c': 3};
```

```
print (d ['c'] )
```

A {'c': 3}

B 2

C 3

D 1

37.下面代码的输出结果是

```
list1 = [1, 2, 3]
```

```
list2 = [4,5,6]
```

```
print(list1+list2)
```

A [4,5,6]

B [1, 2, 3, 4, 5, 6]

C [1, 2,3]

D

[5, 7,9]

38.下面代码的输出结果是

```
str1="k: 1 | k1:2k2: 3 | k3: 4"
```

```
str_list=str1.split(' | ')
```

```
d={}
```

```
for l in str_list:
```

```
    key, value=l.split (':')
```

```

d[key]=value
print(d)
A ['k': '1', 'k1': '2', 'k2': '3', 'k3': '4']
B {'k': '1', 'k1': '2', 'k2': '3', 'k3': '4'}
C {k: 1, k1: 2, k2: 3, k3: 4}
D [k: 1, k1: 2, k2: 3, k3: 4]

```

39. 将以下代码保存成 Python 文件,运行后输出的是

```

li = ['alex','eric', 'rain']
s = "_".join(li)
print (s)
A _alex_eric_rain_
B _alex_eric_rain
C alex_eric_rain_
D alex_eric_rain

```

40. 下面代码的输出结果是

```

li = ["hello", 'se', [ "m", "n"], ["h", "kelly"],'all'],123,446]
print(li [2][1][1])
A h
B kelly
C n
D m

```

41.下面代码的输出结果是

```

a = []
for i in range(2,10):
    count = 0
    for x in range (2, i—1) :
        if i % x == 0:
            count += 1
    if count == 0:
        a.append(i)
print (a)

```

A [3 , 5 , 7 , 9] B [2, 3, 5, 7] C [4, 6, 8 , 9, 10] D [2, 4 , 6, 8]

42.下面代码的输出结果是

```

l1=[1, 2, 3, 2]
l2= ['aa', 'bb','cc','dd','ee']
d= {}
for index in range (len (l1)):
    d [l1[index] ]=l2[index]
print(d)
A {1: 'aa', 2: 'dd', 3: 'cc'}
B {1: 'aa', 2: 'bb', 3: 'cc', 2: 'dd'}
C {1: 'aa', 2: 'bb', 3: 'cc', 2: 'bb'}
D {1: 'aa', 2: 'bb', 3: 'cc'}

```

43. 下面代码的输出结果是

```

i = ['a','b','c']
l = [1, 2, 3]
b = dict(zip (i,l))
print (b)
A 报出异常
B {'a' : 1, 'b' : 2, 'c' : 3}
C 不确定
D {1: 'a', 2: 'd', 3: 'c' }

```

44.下面代码的输出结果是

```

a = [1, 2, 3]

```

```
for i in a[::-1]:
```

```
    print(i, end=" ", " ")
```

A 3, 1,2

B 2,1,3

C 3, 2,1,

D 1, 2,3

45.下面代码的输出结果是

```
L = [1, 2,3,4,5]
```

```
s1 = ','.join(str(n) for n in L)
```

```
print(s1)
```

A [1,2,3,4, 5]

B 1,, 2, ,3,, 4,, 5

C [1,,2,, 3,, 4,, 5]

D

1, 2, 3, 4, 5

46.下面代码的输出结果是

```
a = [9, 6, 4,5]
```

```
N = len(a)
```

```
for i in range(int(len(a) / 2)):
```

```
    a[i],a[N-i-1] = a[N-i-1],a[i]
```

```
print(a)
```

A [9,6,5,4]

B [5,4, 6,9]

C [5,6, 9, 4]

D [9,4,6, 5]

47.下面代码的输出结果是

```
a = [1, 2, 3]
```

```
b = a[:]
```

```
print(b)
```

A []

B [1, 2, 3]

C 0xF0A9

D [3,2,1]

48.下面代码的输出结果是

```
a = [1,3]
```

```
b = [2,4]
```

```
a.extend(b)
```

```
print(a)
```

A [1 , 2 ,3 , 4]

B [1, 3, 2, 4]

C [4 ,3 , 2 ,1]

D

[4 , 2 ,3 , 1]

49.下面代码的输出结果是

```
>>> s = {}
```

```
>>> type(s)
```

A<class 'tuple'>

B <class 'dict'>

C<class 'set'>

D <class 'list'>

50.下面代码的输出结果是

```
>>> s = set()
```

```
>>> type(s)
```

A <class 'tuple'>

B <class 'list'>

C <class 'dict'>

D <class 'set'>

六、文件和数据格式化

1.关于 Python 对文件的处理，以下选项中描述错误的是

A 当文件以文本方式打开时,读写按照字节流方式

B Python 能够以文本和二进制两种方式处理文件

C Python 通过解释器内置的 open() 函数打开一个文件

D 文件使用结束后要用 close()方法关闭,释放文件的使用授权

2.以下选项中,不是 Python 对文件的读操作方法的是

A read

B readline

C readtext

D readlines

3.以下选项中,不是 Python 对文件的打开模式的是

- A 'w' B 'r' C '+' D 'c'

4. 给出如下代码:

```
fname = input("请输入要打开的文件: ")
```

```
fi = open (fname, "r")
```

```
for line in fi.readlines ():
```

```
    print(line)
```

```
fi.close()
```

以下选项中描述错误的是

- A 通过 fi。readlines()方法将文件的全部内容读入一个字典 fi
B 用户输入文件路径,以文本文件方式读入文件内容并逐行打印
C 通过 fi。readlines ()方法将文件的全部内容读入一个列表 fi
D 上述代码中 fi。readlines ()可以优化为 fi

5.关于数据组织的维度,以下选项中描述错误的是

- A 数据组织存在维度,字典类型用于表示一维和二维数据
B 高维数据由键值对类型的数据构成,采用对象方式组织
C 一维数据采用线性方式组织,对应于数学中的数组和集合等概念
D 二维数据采用表格方式组织,对应于数学中的矩阵

6.关于 Python 文件打开模式的描述,以下选项中错误的是

- A 追加写模式 a B 只读模式 r C 覆盖写模式 w D 创建写模式 n

7. 执行如下代码:

```
fname = input("请输入要写入的文件: ")
```

```
fo = open(fname, "w+")
```

```
ls = ["清明时节雨纷纷,", "路上行人欲断魂,", "借问酒家何处有? ","牧童遥指杏花村。"]
```

```
fo.writelines (ls)
```

```
fo。seek(0)
```

```
for line in fo:
```

```
    print (line)
```

```
fo.close()
```

以下选项中描述错误的是

- A *fo。seek(0)这行代码可以省略,不影响输出效果
B * fo。writelines(ls)将元素全为字符串的 ls 列表写入文件
C 代码主要功能为向文件写入一个列表类型,并打印输出结果
D 执行代码时,从键盘输入“清明.txt”,则清明。txt 被创建

8.关于 CSV 文件的描述,以下选项中错误的是

- A *CSV 文件的每一行是一维数据,可以使用 Python 中的列表类型表示
B 整个 CSV 文件是一个二维数据
C *CSV 文件格式是一种通用的、相对简单的文件格式,应用于程序之间转移表格数据
D *CSV 文件通过多种编码表示字符

9.关于 Python 文件的 '+' 打开模式,以下选项中描述正确的是

- A 与 r/w/a/x 一同使用,在原功能基础上增加同时读写功能
B 读模式
C 追加写模式
D 覆盖写模式

- 10.表格类型数据的组织维度是
A 高维数据 B 一维数据 C 多维数据 D 二维数据
11. “键值对”类型数据的组织维度是
A 高维数据
B 二维数据
C 多维数据
D 一维数据
12. 给定列表 `ls = {1, 2, 3, "1", "2", "3"}`, 其元素包含两种数据类型,则 `ls` 的数据组织维度是
A 多维数据 B 二维数据 C 高维数据 D 一维数据
- 13.给定字典 `d = {1:"1", 2:"2", 3:"3"}`,其元素包含两种数据类型, 则字典 `d` 的数据组织维度是
A 二维数据 B 多维数据 C 高维数据 D 一维数据
14. 以下选项中,不是 Python 中文件操作的相关函数是
A `open()` B `read()` C `load()` D `write()`
15. 以下选项中, 不是 Python 中文件操作的相关函数是
A `write()` B `open()` C `readlines()` D `writeline()`
16. 以下选项中, 不是 Python 文件处理。`seek()` 方法的参数是
A 0 B 1 C 2 D -1
- 17.以下选项中, 不是 Python 文件打开的合法模式组合是
A `"r"` B `"a"` C `"+"` D `"w"`
18. 以下选项中, 不是 Python 文件打开的合法模式组合是
A `"a+"` B `"t+"` C `"r+"` D `"w+"`
- 19.以下选项中, 不是 Python 文件打开的合法模式组合是
A `"br+"` B `"wr"` C `""` D `"bw"`
20. 以下选项中, 不是 Python 文件二进制打开模式的合法组合是
A `"b"` B `"x+"` C `"bx"` D `"bw"`
- 21.关于一维数据存储格式问题, 以下选项中描述错误的是
A 一维数据可以采用 CSV 格式存储
B 一维数据可以采用直接相连形成字符串方式存储
C 一维数据可以采用分号分隔方式存储
D 一维数据可以采用特殊符号@分隔方式存储
22. 关于二维数据 CSV 存储问题, 以下选项中描述错误的是
A CSV 文件的每一行表示一个具体的一维数据
B CSV 文件的每行采用逗号分隔多个元素
C CSV 文件不是存储二维数据的唯一方式
D CSV 文件不能包含二维数据的表头信息
23. 以下选项中, 对 CSV 格式的描述正确的是
A CSV 文件以英文特殊符号分隔元素
B CSV 文件以英文逗号分隔元素
C CSV 文件以英文空格分隔元素
D CSV 文件以英文分号分隔元素
24. 关于文件关闭的`.close()`方法,以下选项中描述正确的是
A 文件处理遵循严格的“打开—操作—关闭”模式

- B 文件处理后可以不用.close()方法关闭文件，程序退出时会默认关闭
- C 文件处理结束之后，一定要用.close() 方法关闭文件
- D 如果文件是只读方式打开，仅在这种情况下可以不用。close（）方法关闭文件
- 25.表达式", ".join(ls)中 ls 是列表类型,以下选项中对其功能的描述正确的是
- A 将逗号字符串增加到列表 ls 中
- B 在列表 ls 每个元素后增加一个逗号
- C 将列表所有元素连接成一个字符串，每个元素后增加一个逗号
- D 将列表所有元素连接成一个字符串,元素之间增加一个逗号
- 26.二维列表 ls=[[1,2, 3],[4, 5, 6] ,[7,8, 9]]，以下选项中能获取其中元素 5 的是
- A ls [1] [1] B ls [-2][-1] C ls[4] D ls [-1] [-1]
- 27.二维列表 ls= [[1, 2,3], [4, 5,6] , [7,8,9]]，以下选项中能获取其中元素 9 的是
- A ls [0] [-1] B ls[-1] C ls[-1] [-1] D ls[-2] [-1]
- 28.二维列表 ls= [[1,2,3], [4,5,6],[7,8,9]]，以下选项中能获取其中一个维度的数据是
- A ls[-1] B ls[1][1] C ls [-1] [-1] D ls[-2] [-1]
- 29.列表 ls= [1, 2,3,4, 5, 6, [7, 8,9]]，以下选项中描述正确的是
- A ls 可能是一维列表 B ls 可能是高维列表
- C ls 可能是二维列表 D ls 可能是多维列表
- 30.列表 ls= [[1, 2, 3,4, 5, 6,7,8,9]]，以下选项中描述错误的是
- A ls 可能是一维列表 B ls 可能是二维列表
- C ls 可能是多维列表 D ls 可能是高维列表
- 31.以下文件操作方法中，不能从 CSV 格式文件中读取数据的是
- A readlines B readline C seek D read
- 32.以下文件操作方法中，不能向 CSV 格式文件写入数据的是
- A seek 和 write B writelines C write D writeline
- 33.两次调用文件的 write 方法,以下选项中描述正确的是
- A 连续写入的数据之间默认采用逗号分隔
- B 连续写入的数据之间无分隔符
- C 连续写入的数据之间默认采用空格分隔
- D 连续写入的数据之间默认采用换行分隔
- 34.表达式 writelines (lines) 能够将一个元素是字符串的列表 lines 写入文件,以下选项中描述正确的是
- A 列表 lines 中各元素之间无分隔符
- B 列表 lines 中各元素之间默认采用逗号分隔
- C 列表 lines 中各元素之间默认采用换行分隔
- D 列表 lines 中各元素之间默认采用空格分隔
- 35.关于 open() 函数的文件名，以下选项中描述错误的是
- A 文件名不能是一个目录
- B 文件名对应的文件可以不存在，打开时不会报错
- C 文件名可以是相对路径
- D 文件名可以是绝对路径
36. Python 语句:f = open ()，以下选项中对 f 的描述错误的是
- A 将 f 当作文件对象，f.read () 可以读入文件全部信息
- B 表达式 print(f) 执行将报错
- C *f 是一个 Python 内部变量类型

D *f 是文件句柄，用来在程序中表达文件

37.使用 open() 打开一个 Windows 操作系统 D 盘下的文件，以下选项中对路径的表示错误的是

A D:\PythonTest\ a。txt

B D: //PythonTest//a。txt

C D:\PythonTest\ a.txt

D D: /PythonTest/a。txt

38. 关于下面代码中的变量 x，以下选项中描述正确的是

```
fo = open (fname, "r")
```

```
for x in fo:
```

```
    print(x)
```

```
fo.close ()
```

A 变量 x 表示文件中的一个字符

B 变量 x 表示文件中的全体字符

C 变量 x 表示文件中的一行字符

D 变量 x 表示文件中的一组字符

39.当前程序路径在 D: \PythonTest 目录中，使用 open()打开 D 盘根目录下文件,以下选项中对路径的表示错误的是

A D: \a.txt

B D:\a.txt

C ../a.txt

D .. //a。txt

40 以下选项对应的方法可以用于从 CSV 文件中解析一二维数据的是

A split()

B format()

C exists()

D join()

41.以下选项对应的方法可以用于向 CSV 文件写入一二维数据的是

A split()

B strip()

C exists()

D join()

42.以下选项对应的方法可以辅助用于从 CSV 文件中解析一二维数据的是

A count ()

B strip()

C format()

D center ()

43. 关于 CSV 文件的扩展名,以下选项中描述正确的是

A 可以为任意扩展名

B 扩展名只能是。txt

C 扩展名只能是.csv

D 扩展名只能是。dat

44. 关于文件的打开方式，以下选项中描述正确的是

A 文件只能选择二进制或文本方式打开

B 所有文件都可能以文本方式打开

C 文本文件只能以文本方式打开

D 所有文件都可能以二进制方式打开

45.对于特别大的数据文件，以下选项中描述正确的是

A 选择内存大的计算机，一次性读入再进行操作

B *Python 可以处理特别大的文件，不用特别关心

C 使用 for 。.in。循环,分行读入,逐行处理

D Python 无法处理特别大的数据文件

46. 关于高维数据，以下选项中描述错误的是

A 高维数据只能表达键值对数据

B “键值对”是高维数据的主要特征

C 高维数据可用于表达一二维数据

D 高维数据用来表达索引和数据之间的关系

47.当打开一个不存在的文件时，以下选项中描述正确的是

A 文件不存在则创建文件

B 一定会报错

C 根据打开类型不同，可能不报错

D 不存在文件无法被打开

- 48.关于数据维度,以下选项中描述错误的是
- A 一维数据可能存在顺序，也可以没有顺序
 - B 数据维度包括一二维、多维和高维数据
 - C 所有数据都能用维度方式表示
 - D 图像由于存在长宽，所以图像数据是二维数据
- 49。以下选项中不是文件操作函数或方法的是
- A load
 - B read
 - C writelines
 - D readlines
- 50.对于无序的一维数据，以下选项中描述错误的是
- A 无序一维数据可以采用列表类型来表达
 - B 无序一维数据可以采用字典类型来表达
 - C 无序一维数据可以采用集合类型来表达
 - D 无序一维数据无法利用 Python 语言有效表达

答案:

Python 语法基础

1—5 CCACD	6—10 BBBDB	11—15 ACDCA	16-20 DAACC
21-25 BBAAB	26-30 CABCA	31-35 ACCDA	36-40 AABCA
41-45 DADCC	46-50 ACDCA		

基本数据类型

1-5 CACBC	6-10 ACBCB	11-15 AACBC	16—20 CCABC
21-25 CACBC	26—30 DCBAA	31—35 ADBBA	36—40 BADAC
41-45 DDBBC	46—50 DCACA		

程序的控制结构

1—5 DBACD	6—10 ABDBB	11-15 BCDDD	16—20 CBCAB
21—25 CDADB	26-30 CDCBB	31-35 CCBBA	36—40 DDAAA
41-45 BDBCD	46—50 DDCCB		

函数和代码复用

1—5 DCACA	6-10 CBBBC	11-15 DABCD	16-20 BDAAD
21-25 DABBC	26-30 BDABA	31-35 ABDAB	36-40 DABDB
41—45 BDABD	46—50 DBBAA		

组合数据类型

1—5 CADBC	6—10 DCDBB	11-15 BDDDB	16—20 BBBAC
21—25 ACACA	26—30 ABADC	31—35 ADCAA	36-40 CBBDB
41—45 BABCD	46-50 BBBBD		

文件和数据格式化

1—5 ACDA A	6-10 DADAD	11-15 ADCCD	16-20 DCBBB
21—25 BDBBD	26-30 ACAAD	31—35 CDBAB	36-40 BACBA
41-45 DBADC	46-50 ACDAD		